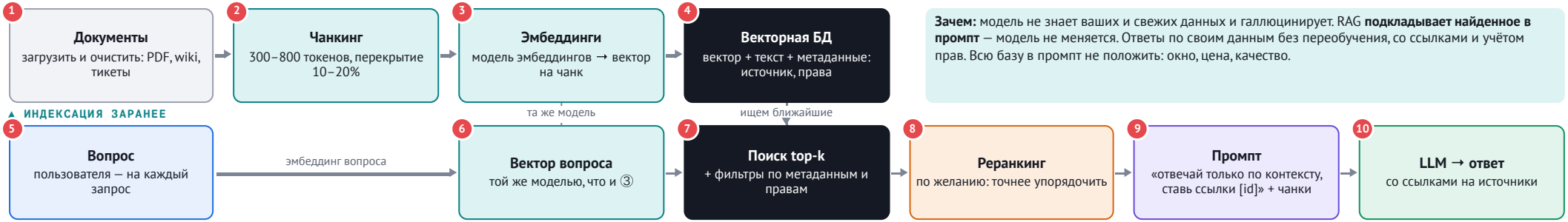
RAG: поиск по своим документам

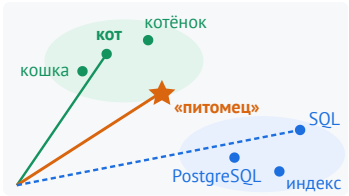
01 RAG-пайплайн по шагам

Retrieval-Augmented Generation: сначала найти нужные куски документов, потом ответить по ним



Зачем: модель не знает ваших и свежих данных и галлюцинирует. RAG **подкладывает найденное в промпт** — модель не меняется. Ответы по своим данным без переобучения, со ссылками и учётом прав. Всю базу в промпт не положить: окно, цена, качество.

02 Эмбеддинги



Эмбеддинг — вектор фиксированной длины (сотни–тысячи чисел). Его строит **отдельная модель эмбеддингов**.

Похожий смысл → близкие векторы, **даже без общих слов**: «питомец» рядом с «кот».

$$\cos(a, b) = a \cdot b \,/\, (|a| \cdot |b|)$$

малый угол → похоже (≈1), большой → не связано

Документы и вопросы — **одной** моделью. Где нужны: семантический поиск, RAG, кластеризация, рекомендации. Эмбеддинг ≠ токен.

03 Векторные базы данных

Векторная БД хранит векторы с текстом и метаданными и быстро находит ближайшие к запросу (top-k). **ANN-индекс HNSW** — почти точный поиск, в разы быстрее полного перебора.

pgvector	расширение Postgres: SQL, транзакции, JOIN с данными
Qdrant	отдельный сервер: быстрые фильтры, масштаб
Chroma	встраиваемая, для прототипов

```
CREATE INDEX ON chunks
  USING hnsw (embedding vector_cosine_ops);
SELECT id, text FROM chunks
WHERE team_id = $2              -- права доступа
ORDER BY embedding <=> $1 LIMIT 5; -- ближайшие
```

`<=>` — косинусное расстояние = 1 − сходство: меньше — ближе. HNSW для `vector` — до 2000 измерений, больше — `halfvec`.

04 Чанкинг: размер и перекрытие

Документ целиком не влезет в окно, а один вектор на длинный текст **размыт**. Режем на чанки, лучше по структуре: заголовки, абзацы.



▨ перекрытие 10–20%: мысль на границе попадает в оба чанка

Слишком крупно
поиск размыт, в окно влезает мало чанков, шум, дороже

Слишком мелко
фраза без контекста, ответ из обрывков

Обычно
300–800 токенов; подбирают по оценке качества

05 Гибридный поиск и реранкинг

BM25 — по словам
коды, артикулы, имена

Векторный — по смыслу
синонимы, перефразы

↓

Объединение (RRF) — по местам в обоих списках

↓

~50 кандидатов

↓

Реранкер (cross-encoder) — читает вопрос и чанк вместе

↓

5 лучших → в LLM

Реранкер точнее, но медленнее — поэтому только по кандидатам. `ts_rank` в Postgres — не BM25.

06 RAG или fine-tuning

RAG — для знаний
документы, факты, часто меняющиеся данные; нужны источники и права доступа

Fine-tuning — для поведения
стиль, формат, узкая задача, короче промпт; новые знания так добавлять плохо

Порядок: **промптинг → RAG → дообучение**, если упёрлись в формат, стиль или цену. Можно сочетать.

RAG — не дообучение: модель не меняется, найденное только подкладывается в промпт.

07 Оценка качества RAG

Тестовый набор
50–200 реальных вопросов: эталон + нужный источник

→

Прогон RAG
найденные чанки + ответ

→

LLM-as-a-judge
оценки по чёткой рубрике + человек на выборке

Поиск (retrieval)
hit rate / recall@k — нужный чанк попал в top-k?
MRR — на каком он месте

Ответ (generation)
faithfulness — всё есть в найденных чанках?
relevance — по вопросу? **correctness** — как эталон?

Меряй поиск и ответ **отдельно**: так видно, где ломается. Любое изменение (чанкинг, модель, промпт) — прогон на **том же наборе**. В проде — фидбэк и мониторинг.

ВОПРОСЫ МЕНТОРА → Эмбеддинг 02 · Векторная БД 03 · RAG и какую проблему решает 01 · RAG-пайплайн по шагам 01 · Чанкинг: крупные и мелкие 04 · RAG или fine-tuning 06 · Оценка RAG 07